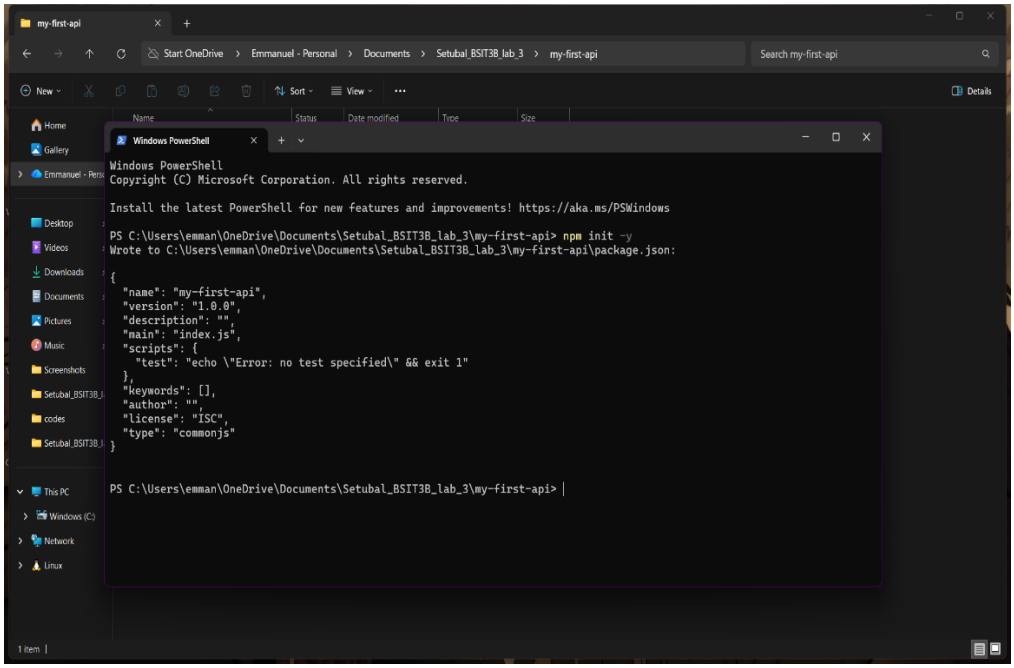
Arzel Andrei Israel
John Christian Setubal
Lynxie Jagonob
John Walter Pediangco
Emmanuel Siastres

Lab 3. Setup REST API Server

Objective: Shift from being the "Client" to being the "Server." Set up a basic Node.js API that handles HTTP verbs (GET and POST).

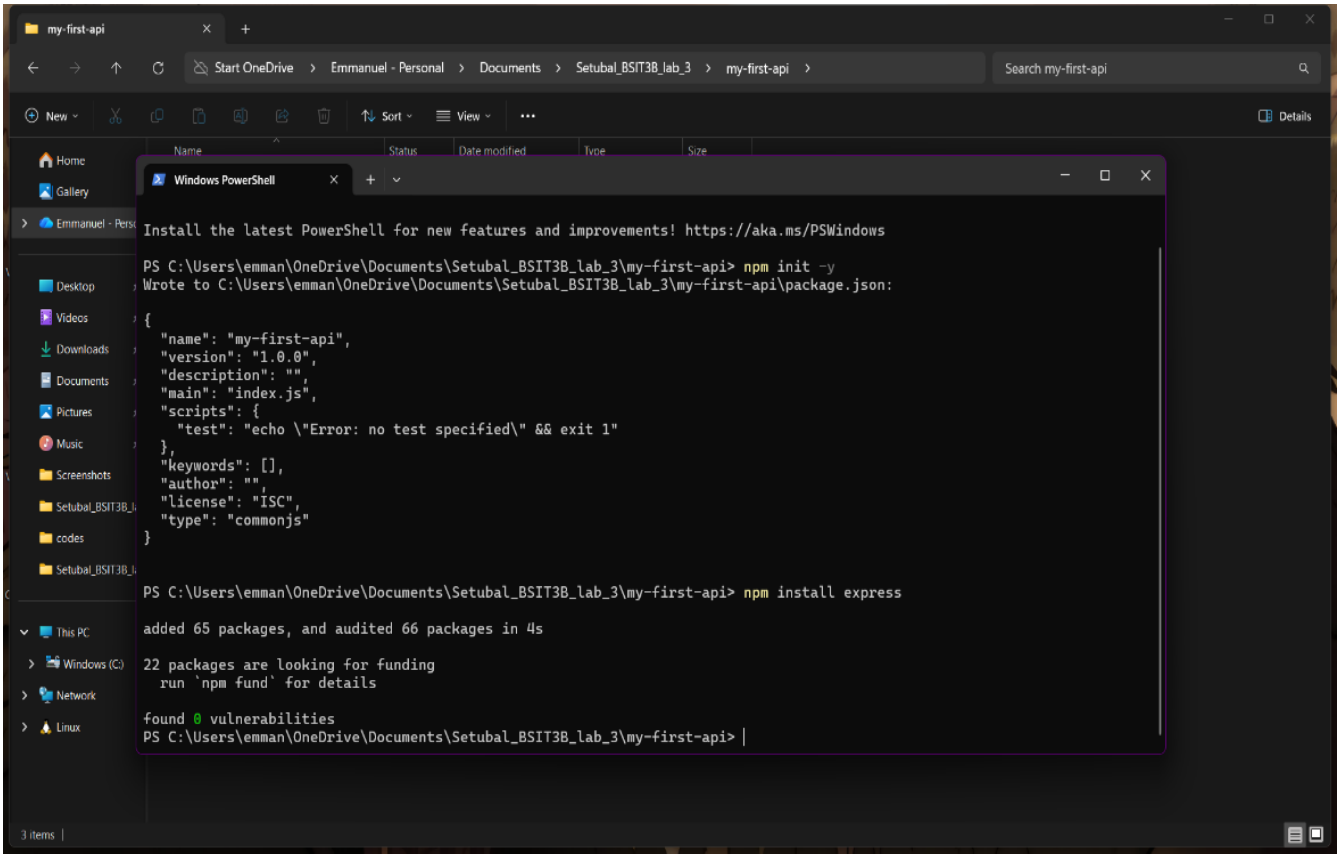
Instructions

1. Setup Your Environment
   - Create a new folder named my-first-api (or rename something else) and navigate into it using your terminal.
   - Initialize a new Node.js project by running npm init -y.



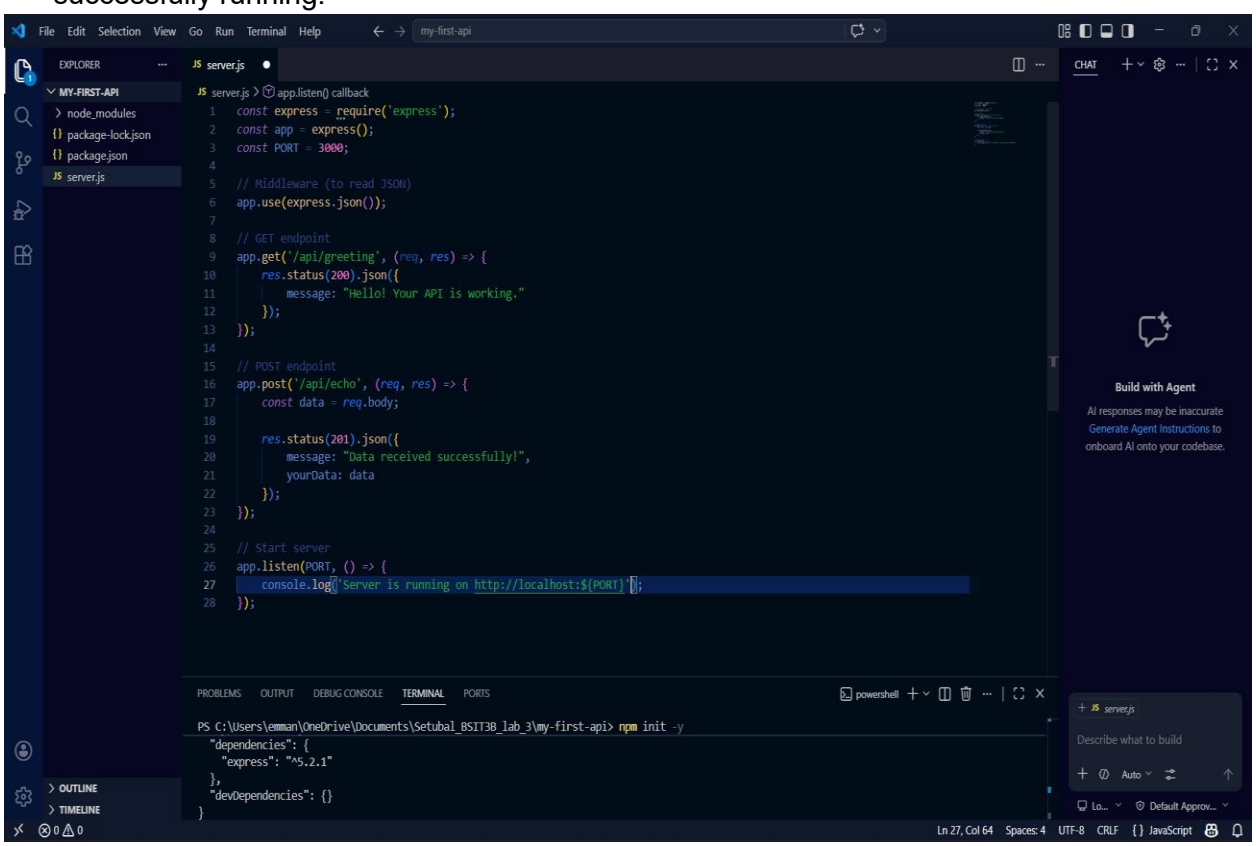
2. Install Express

- Express is a web framework that makes building APIs in Node.js incredibly simple. Install it by running
  - npm install express

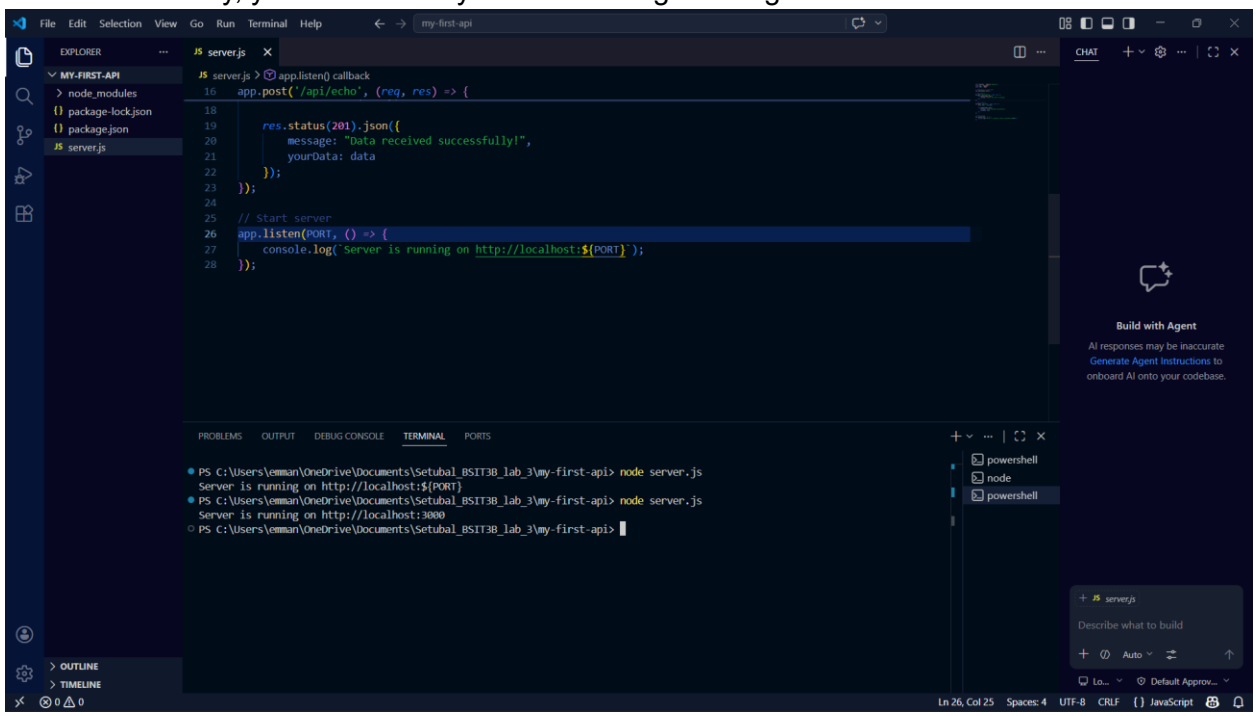


3. Build the Server

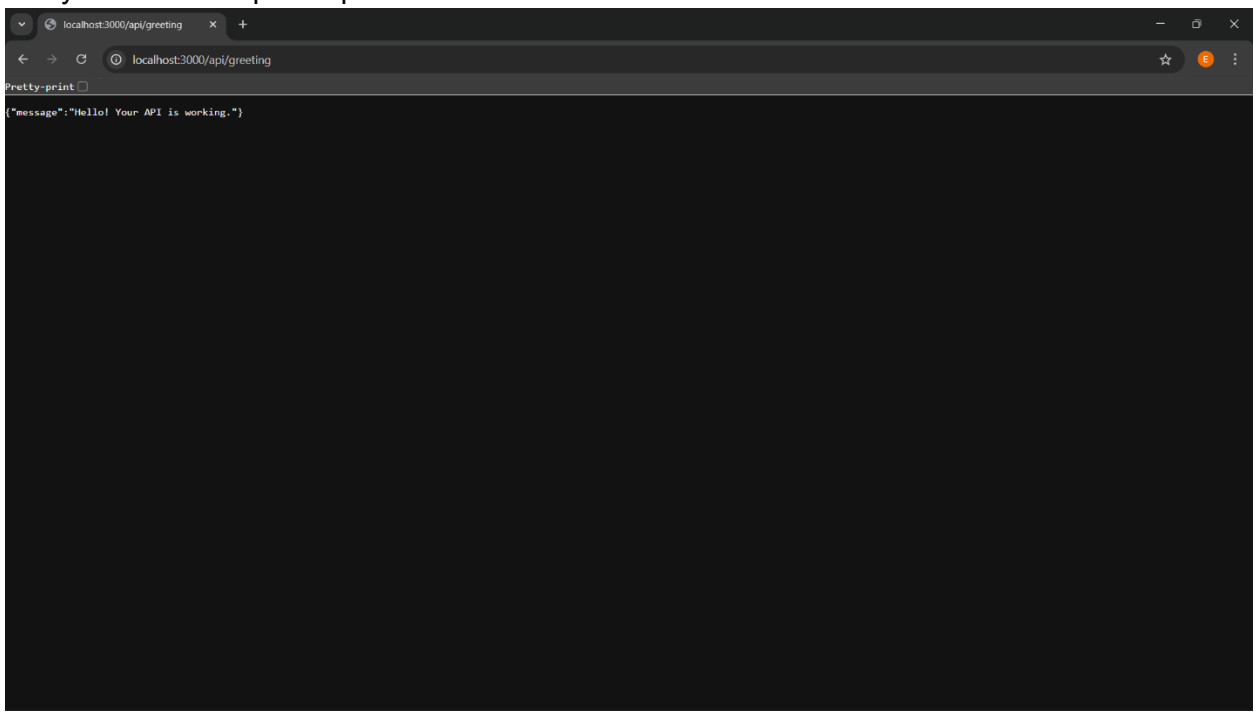
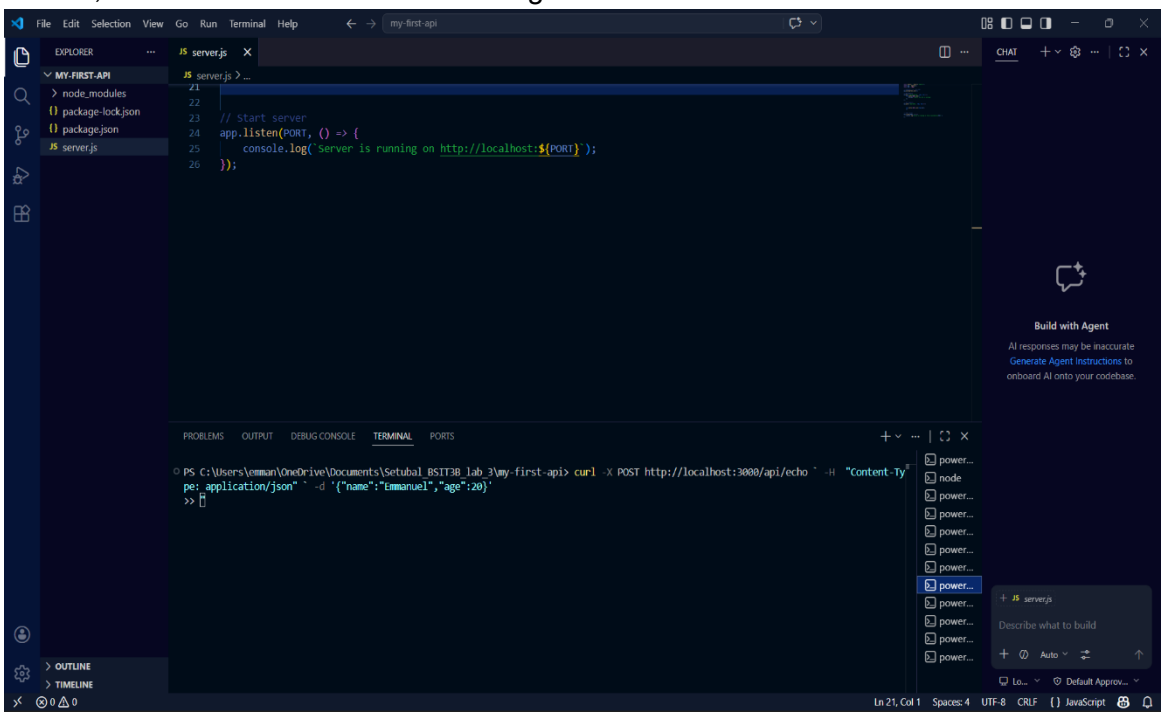
- Create a file named server.js (again, you can rename this all you want). Use the following hints to write the server logic yourself
  - Initialization: Require the express module, create your app instance, and define a constant for your PORT (e.g., 3000).
  - Middleware: Before you write your endpoints, you need to tell Express how to read JSON payloads. Use app.use() to implement the built-in express.json() middleware.
  - The GET Endpoint (Read Data): Create a route that listens for GET requests at the endpoint /api/greeting. Inside the callback function (req, res), use the response object to send a 200 status code and a JSON object with a welcome message (e.g., "Hello! Your API is working.").
  - The POST Endpoint (Receive Data): Create a route that listens for POST requests at the endpoint /api/echo.
    - Hint: You can access the data the client sent you via req.body. Store this in a variable.
    - Use the response object to send back a 201 status code (which means "Created") and a JSON object that includes both a success message and the data the user just sent you.
  - Start Listening: Finally, tell your app to listen on your defined PORT. Include a console.log in the callback function so your terminal tells you when the server is successfully running.



4. Run the Server
   - Type (node+your js file = e.g. node name.js, node sever.js) in your terminal. If you wrote the code correctly, you should see your console log message



5. Test Endpoints
   - Test GET: Open a web browser and go to http://localhost:3000/api/greeting. You should see your JSON response printed on the screen.



- Test POST: Open a new terminal window and use curl to send a JSON payload to your echo endpoint.
    - Hint: You will need to use -X POST, set the -H "Content-Type: application/json" header, and use -d to send a JSON string. See if it echoes back